

Task 0 – JavaScript Refresher

This practice is a JavaScript refresher before we start working with React.

This is **not a graded assignment** and you do not have to submit it.

If you want to receive the bonus point, complete **all tasks** and submit your solution before the deadline as a nicely organized **HTML + CSS + JavaScript page**. Show the results of the tasks on the page and add short explanations or your own insights about what you learned or noticed.

I will review these submissions and grade them for the bonus point.

1 – Variables and Data Types

Create variables for a student: name, age, active status, courses, and address.

- Print the values and their types.
- Add examples of null and undefined.
- Identify primitive and reference values.
- Create a sentence using a template literal.

Answer briefly:

- What is the difference between let and const?
 - What does typeof null return?
 - What are JavaScript primitive types?
-

2 – Arrays

Use:

[3, 7, 2, 10, 5]

Using map, filter, find, reduce, and includes:

- Multiply every number by 2.

- Get numbers greater than 5.
- Find the first number greater than 5.
- Calculate the sum.
- Check whether 10 exists.

Do not modify the original array.

3 – Arrays of Objects

Create:

- Anna – id: 1, grade: 85
- John – id: 2, grade: 62
- Sara – id: 3, grade: 91
- Mike – id: 4, grade: 55

Find or create:

- Students with grade ≥ 70 .
- An array of student names.
- Student with id = 3.
- Student with the highest grade.
- Average grade.
- A new array where every student has passed: true/false.

Do not modify the original objects.

4 – Objects

Create a user with id, name, age, and an address containing city and street.

- Read the name and city.
- Change the age.
- Add an email.
- Remove the street.
- Get name and age using destructuring.
- Get city using nested destructuring.
- Rename name to userName during destructuring.

5 – Values and References

Start with:

```
original = { name: "Alice", score: 10 }
```

```
copy = original
```

Change `copy.score`.

- Check what happens to `original`.
- Explain why.
- Make a copy using the spread operator.
- Change the copy without changing the original.

Then try:

```
user = { name: "Alice", address: { city: "Almaty" } }
```

Copy it using spread and change the city.

- What happens to the original?
 - Why?
 - Find a way to copy the nested address correctly.
-

6 – Functions

Create:

```
isEven(number)
```

```
getFullName(firstName, lastName)
```

```
calculatePrice(price, quantity)
```

```
calculateDiscount(price, percent)
```

```
getMax(a, b)
```

Write at least one using normal function syntax, then rewrite it as an arrow function.

7 – Functions as Values

Create:

`add(a, b)`

`multiply(a, b)`

Then create:

`calculate(a, b, operation)`

It should work like this:

`calculate(5, 3, add) → 8`

`calculate(5, 3, multiply) → 15`

Explain briefly:

- Can functions be stored in variables?
- Can functions be passed to other functions?
- What is the difference between `add` and `add()`?

8 – Scope

Create:

`const message = "global"`

Create a function with another `message = "function"`.

Inside it, create an if block with another `message = "block"`.

Print `message` at each level and observe the result.

Also create variables using `var`, `let`, and `const` inside a block and try accessing them outside.

Explain briefly:

- Global scope
 - Function scope
 - Block scope
 - Difference between var, let, and const
-

9 – Closure

Create:

```
createCounter()
```

It should keep its own count variable and return a function that increases it.

Expected result:

```
counter() → 1
```

```
counter() → 2
```

```
counter() → 3
```

Create another counter and check whether it has its own count.

Then create:

```
createAdder(value)
```

Example:

```
const addFive = createAdder(5)
```

```
addFive(10) → 15
```

```
addFive(20) → 25
```

Explain briefly why the inner function can still access variables from the outer function.

10 – Destructuring, Spread and Rest

Use:

```
numbers = [10, 20, 30, 40]
```

Get the first two values using destructuring.

Use:

```
user = { id: 1, name: "Anna", age: 21 }
```

Get name and age using destructuring.

Then:

- Add 50 to a new copy of the numbers array.
- Create a new user with age 22.
- Add an email without modifying the original user.
- Combine two arrays using spread.

Create a function:

```
sum(...numbers)
```

It should accept any number of arguments.

```
sum(1, 2) → 3
```

```
sum(1, 2, 3, 4) → 10
```

Explain the difference between spread and rest.

11 – Optional Chaining and Default Values

Create user objects where some users have an address and some do not.

Try:

```
user.address.city
```

Then solve the problem using optional chaining ?..

Use nullish coalescing ?? to display:

```
"City not specified"
```

when the city does not exist.

Compare `||` and `??` using:

`0`, `""`, `false`, `null`, `undefined`

Explain what you notice.

Final Task

Create an array with at least **5 students**.

Each student should have:

`id`, `name`, `age`, `grades`

where `grades` is an array of numbers.

Create:

`getAverage(grades)`

`getStudentAverage(student)`

`getPassedStudents(students)`

`getStudentNames(students)`

`findStudent(students, id)`

`getTopStudent(students)`

Finally, create a new array containing:

`{ id, name, average, passed }`

for every student.

Try to solve the task using modern JavaScript and without modifying the original data.

References

MDN JavaScript Guide

[JavaScript Guide](#)

JavaScript.info – JavaScript Fundamentals

[JavaScript Fundamentals](#)

JavaScript.info – Objects

[Objects](#)